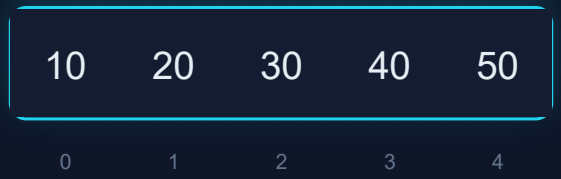
FOUNDATIONS

Data Structures & Algorithms

Week 03 — Arrays

“The simplest data structure can teach us some of the most important ideas in DSA.”



WEEK 01,02 · RECAP

Where Are We?

1. **Problem** — something we need to solve
↓
2. **Algorithm** — a step-by-step plan
↓
3. **Data Structure** — how we organize the data
↓
4. **Efficiency** — how fast / how much memory
↓
5. **Big-O** — the language of cost

Last week we asked *"how do we measure cost?"*

Today we ask *"what do we put the data **in**?"*

We need our first major data structure.

→ **THE ARRAY**

THE PROBLEM

Imagine 1,000 Student Marks

A professor wants to record marks for an entire class:

STUDENT 1	78
STUDENT 2	91
STUDENT 3	65
…	…
STUDENT 1000	88

The ridiculous approach:

marks.cpp

```
int mark1
int mark2
int mark3
int mark4
// ... 996 more lines ...
int mark1000
```

No. Writing 1,000 separate variables is not engineering — it is torture. torture.

There must be a better way. →

THE PROBLEM

The Need for a Collection

We need a single container that gives us all of this at once:

Many
Many values

Hold 10, 1,000, or 1,000,000 numbers under one name.

Type
Same type

All elements share one data data type — all `int`, all `double`…

Group
One logical group

“Marks of students in this class” — one concept, one name.

Access
Easy access

Get or set any element by its its position instantly.

Walk
Easy traversal

Loop over every element with a with a simple `for` loop.

A data structure that satisfies all five requirements is called an **ARRAY**.

What Is an Array?

> An array is a **collection of elements of the same type** stored in a **contiguous block of memory**.

Three ideas, fused together:

[ ]

Same type

Every element has the same data type — e.g. all `int`.

N

Multiple elements

A fixed number of elements share a single name.

[. .]

Contiguous memory

Elements sit in back-to-back memory locations.

`Same type` + `Multiple elements` + `Contiguous memory` = **ARRAY**

WHAT IS AN ARRAY?

The Simplest Array

declaration.cpp

`int`

Data type

Every element will be an integer (4 bytes each).

`marks`

Array name

One name we use to refer to the whole collection. collection.

`[5]`

Number of elements

This array has space for exactly 5 integers.

Declared, but not yet filled — five empty cells:

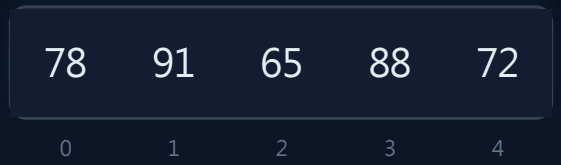


INITIALIZATION

Creating an Array With Values

initialize.cpp

```cpp
int marks [5] =  { 78, 91,  65,  88,  72 };
```



Five elements → indexes **0 through 4**. The first element is at index **0**, the last at index **4**.

The number in `[]` is the **capacity**. The values in `{}` are the **initial contents**.

ZERO-BASED INDEXING

Why Does Indexing Start at 0?

The index is an **offset from the beginning** — how many steps away from the first element.

index 0

→ first element

zero steps from the start

index 1

→ second element

one step from the start

index 2

→ third element

two steps from the start

Analogy: **the first position is zero steps away from the beginning.**

ZERO-BASED INDEXING

Index vs Position

A common source of beginner confusion — let's lay them side by side.



Humans often count positions from **1** — "the 3rd student".

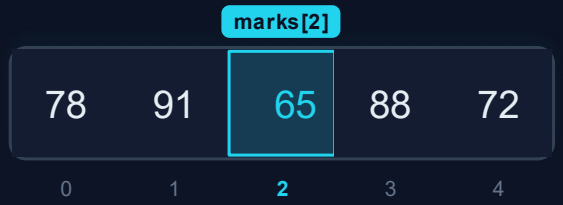
Programming arrays commonly use indexes starting from **0** — marks[2] is the 3rd element.

ACCESSING ELEMENTS

Access an Element

access.cpp

```
Cout  <<marks [2];
```



Question: **What will this print?**

Answer: **65**

The element at index 2 is the *third* value — which is 65.

ACCESSING ELEMENTS

Update an Element

update.cpp

```
marks [2 ] =  70;
```



Array elements can be **updated directly** using their index — no shifting, no extra work.

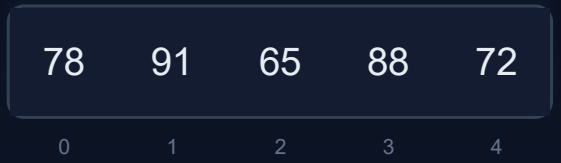
CONTIGUOUS MEMORY

Where Are Array Elements Stored?

Our array marks[5] = {78, 91, 65, 88, 72} sits in memory **back-to-**

For a typical 4-byte `int`, consecutive elements occupy consecutive memory addresses.

1000	→	78
1004	→	91
1008	→	65
1012	→	88
1016	→	72



Each cell uses **4 bytes** (the size of an `int`).

The next element is exactly **4 bytes** after the previous one — no no jumps.

SCONTIGUOUS MEMORY

Why Contiguous Memory Matters

Because elements are arranged sequentially, the computer can **calculate where an element is** from its index from its index — without searching.

Start address of array + (index × element size) = Address of any element

For marks[3]

address = 1000 + (3 × 4) = 1012

Result

The CPU jumps directly to 1012 and reads 88.

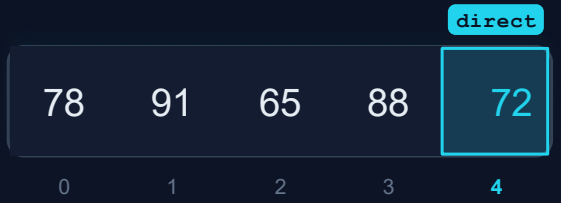
This is the reason array access is **very fast**.

ARRAY ACCESS = O(1)

Direct Access

direct.cpp

```
Cout << marks [4];
```



We don't need to inspect marks[0], marks[1], marks[2], marks[3]...

We directly jump to marks[4].

O(1)

ARRAY ACCESS = O(1)

Why Is Array Access O(1)?

Array → Index → Direct location → Element

1

The CPU reads the element at a calculated address — a **single** step.

n

The size of the array **does not** change how many steps it takes.

The size of the array **does not determine** how many elements we inspect for direct indexed access.

TRAVERSAL

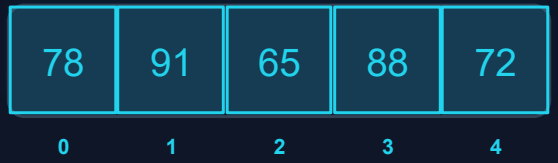
What If We Want Every Element?

What if we want to print every mark?

traverse.cpp

```
for  (int i= 0;  i<  5;  i++)
    Cout<<  marks [i]<< endl
```

The loop walks through the array, one index at a time:



TRAVERSAL

How Traversal Works

I = 0 → **marks[0]**

I = 1 → **marks[1]**

I = 2 → **marks[2]**

I = 3 → **marks[3]**

I = 4 → **marks[4]**

5 elements → 5 accesses.

For n elements → n accesses.

Therefore : **O(n)**

Traversal cost grows **linearly** with array size.

USER INPUT

Reading Values Into an Array

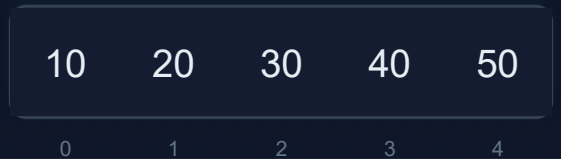
input.cpp

```
for (int i = 0; i < 5; i++) {
    std::cout << "Enter number " << i + 1 << ": ";
    std::cin >> numbers[i];
}
```

Iteration walkthrough — input: 10 20 30 40 50

I = 0	cin → numbers[0] = **10**
I = 1	cin → numbers[1] = **20**
I = 2	cin → numbers[2] = **30**
I = 3	cin → numbers[3] = **40**
I = 4	cin → numbers[4] = **50**

> The loop reads one value per iteration and places it at index `i`.



BASIC ARRAY OPERATIONS

The Array Toolbox

Six operations form the heart of working with arrays — we will analyze the cost of each one.

[i]

Access

Read element at an index.

[i]=

Update

Write a value at an index.

↻

Traverse

Visit every element in order.

Search

Find an element by value.

Insert

Add a value at a position.

Delete

Remove a value at a position.

We will **analyze the cost** of each operation — that is the spirit of DSA.

SEARCHING

Find a Student Mark

Given the array below, find the index of **88**.



How can we find it?

A natural strategy:

1. Look at the first element.
2. Compare it with 88.
3. If it matches → stop.
4. If not → move to the next element.
5. Repeat until found or until we run out of elements.

> This is **Linear Search** — the simplest search algorithm.

SEARCHING

Linear Search — Algorithm

1. Start at **index 0**.

↓

2. Compare the current element with the target.

↓

3. If equal, **return the index** — we found it.

↓

4. Otherwise, move to the **next** element.

↓

5. Continue until found **or** until the array ends (return –1).

> We inspect elements one at a time, in order — hence the time complexity is **O(n)**.

SEARCHING

Linear Search — C++

linear_search.cpp

```cpp
int target = 88;
int index = -1;        // "not found"

for (int i = 0; i < 5; ++i) {
    if (marks[i] == target) {
        index = i;
        break;          // stop early
    }
}

cout << index;
```

Line 1: We're searching for 88.

Line 2: Start with index = −1 as the “not found” marker.

Lines 4–8: Loop through every element. If a match is found, save the index and `break`.

Output: 3 — the index where 88 lives.

SEARCHING

Linear Search — Complexity

Best case

Target is the first element.

O(1)

Worst case

Target is last — or not present.

O(n)

Average

Typically halfway through.

O(n)

Dry run — find 40 in [10, 20, 30, 40, 50]:

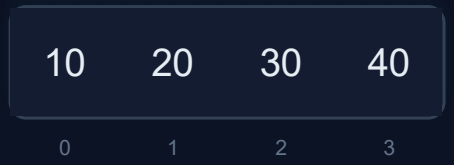
i	numbers[i]	Match?
0	10	No
1	20	No
2	30	No
3	**40**	**Yes ✓**

Searching an **unsorted** array generally requires **checking elements one by elements one by one** — there is no shortcut.

INSERTION

The Insertion Problem

Original array

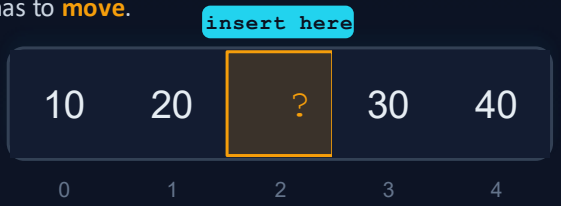


Insert **25** between 20 and 30.

Can we simply place 25 there?

No. The cell at index 2 is already occupied by 30.

Something has to **move**.



INSERTION

Shifting Elements



We shift **left to right**, one cell at a time, opening a gap for the new value.

INSERTION

Array Insertion — C++

```
6       int numbers[10] = {10, 20, 30, 40};
7       int size = 4;
8       int position = 2;
9       int value = 25;
10
11      for (int i = size; i > position; i--) {
12          numbers[i] = numbers[i - 1];
13      }
14
15      numbers[position] = value;
16      size++;
17
18      for (int i = 0; i < size; i++) {
19          cout << numbers[i] << " ";
20      }
21
```

Line 11–12: The loop walks *backward* — from `size` down to position position + 1 — copying each element one step to the right.

Line 15: Now that index 2 is free, drop the new value in.

Line 16: Logical size goes from 4 → 5.

We never shrink or grow the physical array — we only manage the **logical** the **logical size**.

INSERTION

Insertion Complexity

Beginning

All n elements must shift right.

O(n)

Middle

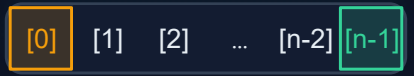
About half the elements shift.

O(n)

End *

No shift — just place & increment.

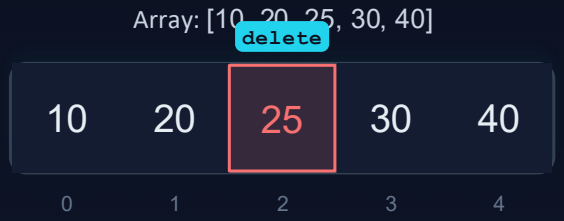
O(1)



* Assumes the array has **unused capacity** and focuses only on shifting cost. Inserting into a fixed-size array also requires **capacity management**.

DELETION

The Deletion Problem



What happens to the gap?

> We can't just leave an empty cell — that would break our our array's order.

Elements after the deleted position must **shift left** to fill the gap.

> Deletion is the **mirror image** of insertion.

DELETION

Shifting After Deletion



We shift **left-to-right**, copying each next element into the current cell.

DELETION

Array Deletion — C++

```
6     int numbers[10] = {10, 20, 30, 40, 50};
7     int size = 5;
8     int position = 2;
9
10    for (int i = 0; i < size; ++i) {
11        cout << numbers[i] << " ";
12    }
13    cout << endl;
14
15    if (position < 0 || position >= size) {
16        cout << "Invalid position!" << endl;
17    } else {
18
19        for (int i = position; i < size - 1; ++i) {
20            numbers[i] = numbers[i + 1];
21        }
22
23        size--;
24
25        cout << "Array after deletion: ";
26        for (int i = 0; i < size; ++i) {
27            cout << numbers[i] << " ";
28        }
```

Lines 10–11: The loop walks *forward* from `position` to size - 2, copying the next element into the current cell.

The element at `position` is overwritten by the next one — and so on, down the line.

Line 23: Logical size decreases by one. The "extra" tail value is outside the logical range.

The physical array capacity **does not shrink** — only the **logical size size** changes.

DELETION

Deletion Complexity

Beginning

All n−1 elements shift left.

O(n)

Middle

About half the elements shift.

O(n)

End

No shift — just decrement size.

O(1)

The main reason: **shifting elements** to fill the gap left by the deleted value.

Deletion mirrors insertion — both pay the **shifting cost** when operating in the middle.

Array Operation Complexity

COMPLEXITY TABLE

Operation	Typical Complexity
Access by index	O(1)
Update by index	O(1)
Traversal	O(n)
Linear search	O(n)
Insert at beginning	O(n)
Insert in middle	O(n)
Insert at end *	O(1)
Delete beginning	O(n)
Delete middle	O(n)
Delete end	O(1)

* assumes there is unused capacity and focuses on shifting cost.

Reference slide — come back to this whenever you forget why an array operation is fast or slow.

REAL-WORLD ARRAY EXAMPLES

Where Do We Use Arrays?

- Student marks
- Daily temperatures
- Product prices
- Monthly sales
- Exam scores
- Pixel values
- Game scores
- Sensor readings
- Seat numbers
- Inventory counts

Arrays are especially useful when data is **naturally indexed** and accessed **frequently by position**.

REAL-WORLD PROBLEM 1

Student Marks

Problem. A class has 10 students. Store their marks and compute:

↑	↓	$\bar{x}$	≥80
Highest mark	Lowest mark	Average	Count above 80

Input array (sample marks):

78	91	65	88	72	84	57	93	69	82
0	1	2	3	4	5	6	7	8	9

Try first. Sketch the algorithm in plain English before writing any C++.

PROBLEM 1 — THINK

Highest Mark

Find the **highest mark** in the array.

MARKS[10]

78	91	65	88	72	84	57	93	69	82
0	1	2	3	4	5	6	7	8	9

THINK FIRST
How can we find the largest value?

1. Start with the first mark as the current highest.
2. Compare every remaining mark with it.
3. If a mark is larger, update the highest.
4. Continue until the array ends.

PROBLEM 1 — FLOWCHART

Highest Mark — Algorithm



MARKS[10]

0	1	2	3	4	5	6	7	8	9
78	91	65	88	72	84	57	93	69	82

PROBLEM 1 — PSEUDOCODE

Highest Mark — Pseudocode

```
1   BEGIN
2       highest ← marks[0]
3       i ← 1
4
5       WHILE i < 10 DO
6           IF marks[i] > highest THEN
7               highest ← marks[i]
8           END IF
9           i ← i + 1
10      END WHILE
11
12      DISPLAY highest
13  END
```

Why this works: We keep one running running value as the current highest and highest and update it whenever a larger larger value appears.

The loop visits index 1 through 9. Index 0 is already stored in `highest`, `highest`, so we don't compare it again. again.

Result: At the end, `highest` holds the largest value in the array. For our marks, that is 93.

PROBLEM 1 — C++ SOLUTION

Highest Mark — C++

```
6     int marks[10] = {78, 91, 65, 88, 72, 84, 57, 93, 69, 82};
7
8     int highest = marks[0];
9
10
11    int i = 1;
12
13
14    while (i < 10) {
15
16        if (marks[i] > highest) {
17            highest = marks[i];
18        }
19        i = i + 1;
20    }
21
22    cout << "Highest mark: " << highest << endl;
```

✦ OUTPUT

✓ Highest mark = 93

ⓘ The `for` loop runs from `i = 1` to `i = 9`. the `if` updates `highest` whenever a larger value is found.

◆ Notice how the C++ code mirrors the pseudocode almost line by line — same skeleton, just with curly braces and semicolons.

PROBLEM 2 — THINK

Lowest Mark

GÖÕÑ POÑ ÕÕR ÑCŒPÖ MØÔ ÕÕ POÑ MØØVŘB

MARKS[10]

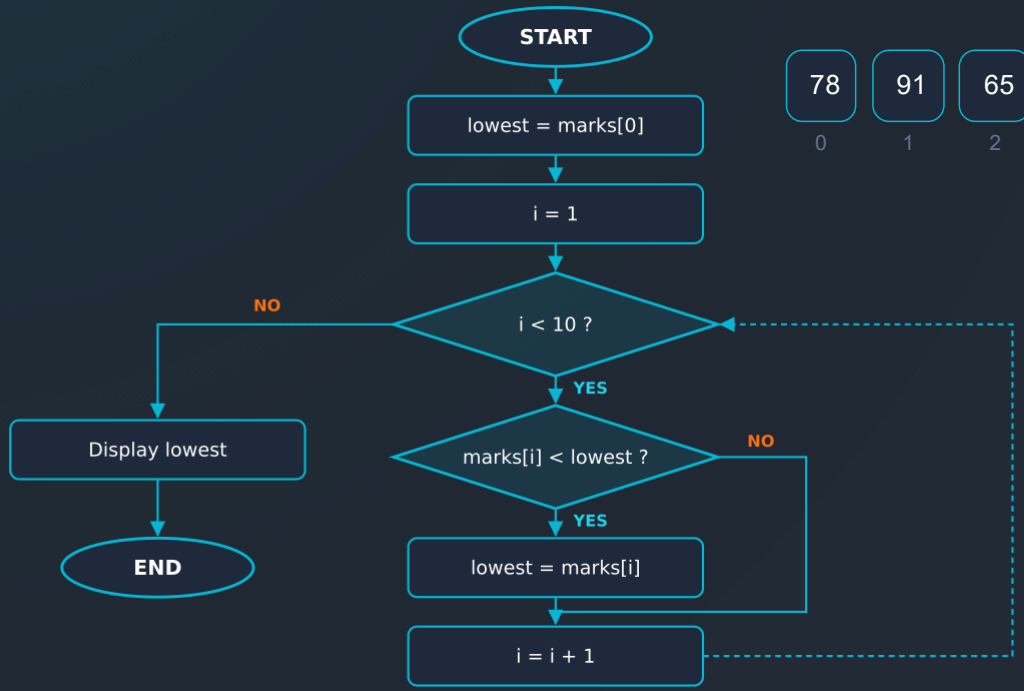
78	91	65	88	72	84	57	93	69	82
0	1	2	3	4	5	6	7	8	9

THINK FIRST
GQŐR ŃMŌ R Ñ MÖÕÑ POÑ CË MÕÕNCŒ QMÕÞÑË

1. Start with marks[0] as the current lowest.
2. Compare every remaining mark.
3. If a smaller value is found, update lowest.
4. F ÖÕRÕÞÑ ÞŌRÕÕPOÑ MØØVŘ ÑÕÑCB

PROBLEM 2 — FLOWCHART

Lowest Mark — Algorithm



MARKS[10]

0	1	2	3	4	5	6	7	8	9
78	91	65	88	72	84	57	93	69	82

PROBLEM 2 — PSEUDOCODE

Lowest Mark — Pseudocode

```
1   BEGIN
2
3       lowest ← marks[0]
4       i ← 1
5
6
7       WHILE i < 10 DO
8           IF marks[i] < lowest THEN
9               lowest ← marks[i]
10          END IF
11          i ← i + 1
12      END WHILE
13
14      // Output the result
15      DISPLAY lowest
16  END
```

⇄ The logic is almost the same as finding the the highest value. Only the comparison changes: > becomes <

Same loop bounds (1 to 9), same update pattern, same display step.

◆ [illegible]

PROBLEM 2 — C++ SOLUTION

Lowest Mark — C++

```cpp
    int marks[10] = {78, 91, 65, 88, 72, 84, 57, 93, 69, 82};

    int lowest = marks[0];

    int i = 1;

    while (i < 10) {

        if (marks[i] < lowest) {
            lowest = marks[i];
        }
        i = i + 1;
    }

    cout << "Lowest mark: " << lowest << endl;
```

✦ OUTPUT

✓ Lowest mark = 57

⇄ Differs from the previous C++ by only two two tokens: `highest` → `lowest`, and `>` → `<`.

◆ The smallest mark, 57, lives at index 6. The loop finds it on the seventh iteration.

PROBLEM 3 — THINK

Ė QÑØMNJÑÌ MØÔ

Calculate the **average mark** of all students.

MARKS[10]

78	91	65	88	72	84	57	93	69	82
0	1	2	3	4	5	6	7	8	9

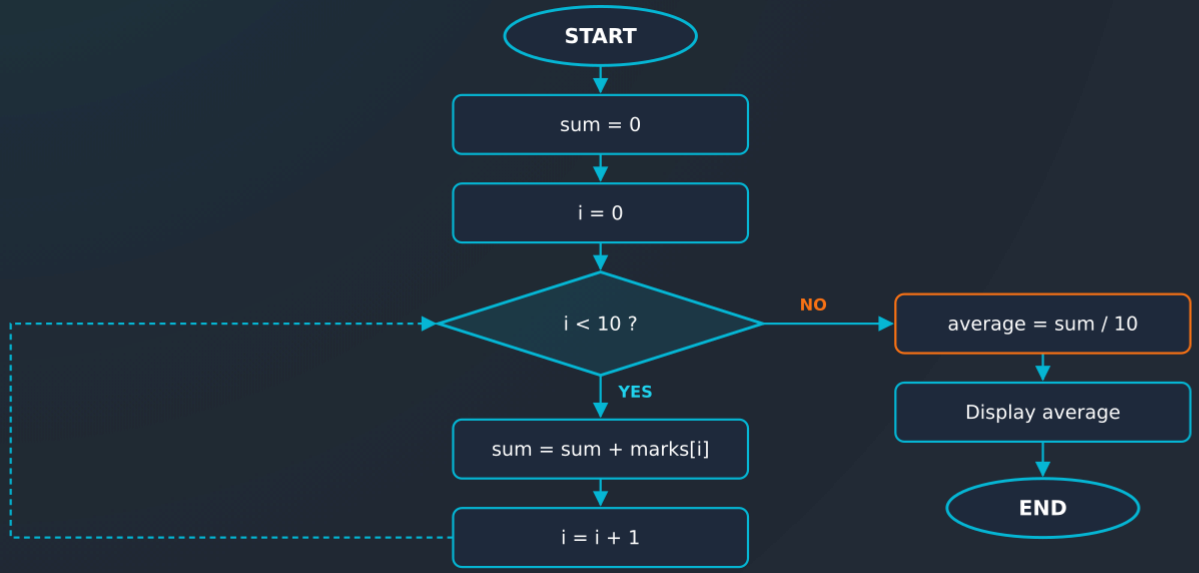
Ì ĢHÍ ĦĞHĪĬ
How do we calculate an average from a list of numbers?
numbers?

1. Start sum at 0.
2. Visit every mark.
3. Add each mark to sum.
4. Divide the final sum by the number of students.

```
Average = Sum / Number of Students
```

PROBLEM 3 — FLOWCHART

Average Mark — Algorithm



PROBLEM 3 — PSEUDOCODE

Average Mark — Pseudocode

```
1   BEGIN
2       sum ← 0
3       i ← 0
4       WHILE i < 10 DO
5           sum ← sum + marks[i]
6           i ← i + 1
7       END WHILE
8
9       average ← sum / 10
10
11      DISPLAY average
12  END
```

ⓘ `sum` keeps accumulating the marks as we move move through the array. Each iteration adds adds one more value.

✦ The division happens **after** the loop — only then do we know the total sum.

`sum: 0 →78 →169 →234 →322 →…`

PROBLEM 3 — C++ SOLUTION

Average Mark — C++

```cpp
    int marks[10] = {78, 91, 65, 88, 72, 84, 57, 93, 69, 82};
    int sum = 0;

    int i = 0;


    while (i < 10) {
        sum = sum + marks[i];
        i = i + 1;
    }

    double average = static_cast<double>(sum) / 10;


    cout << "Average mark: " << average << endl;
```

✦ OUTPUT

✓ Average = 77.9

⚠ Critical: The cast `static_cast<double>(sum)` forces floating-point division. Without it, sum / 10 10 would do integer division and give 77. 77.

◆ Loop runs from i = 0 to i = 9 — every element is added, including the first.

PROBLEM 4 — THINK

Count Above 80

Count how many students scored **80 or above**.

MARKS[10] · HIGHLIGHTS: MARKS ≥ 80

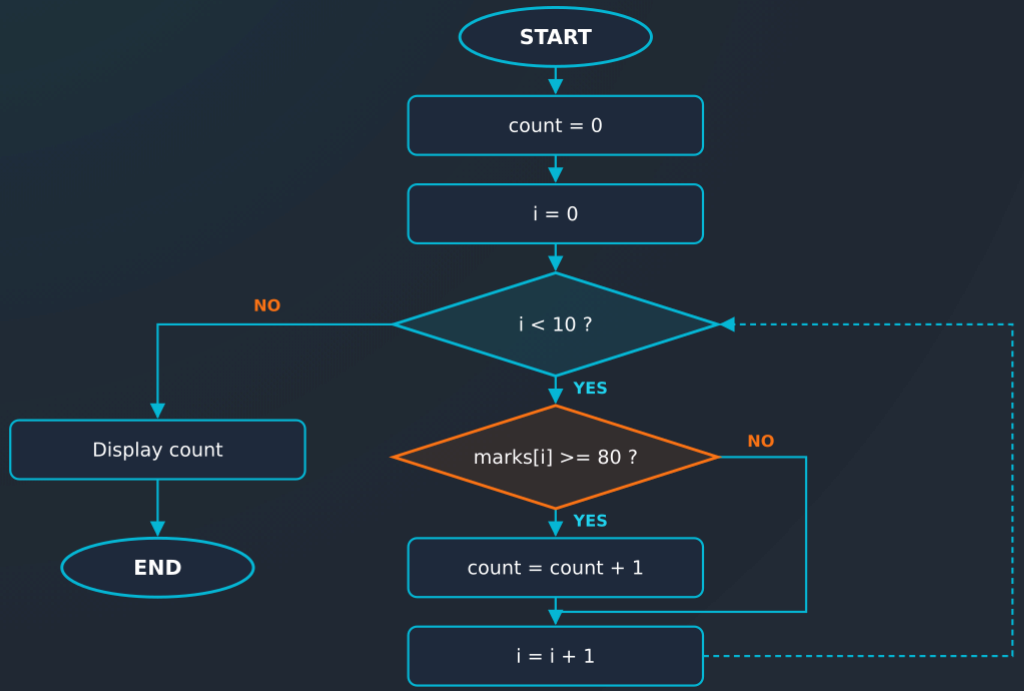
0	1	2	3	4	5	6	7	8	9
78	91	65	88	72	84	57	93	69	82

THINK FIRST
How can we count how many marks satisfy a condition?
condition?

1. Start count at 0.
2. Visit every mark.
3. Check whether mark >= 80.
4. If true, increase count.
5. Continue until the array ends.

PROBLEM 4 — FLOWCHART

Count Above 80 — Algorithm



PROBLEM 4 — PSEUDOCODE

Count Above 80 — Pseudocode

```
1   BEGIN
2       count ← 0
3       i ← 0
4
5       WHILE i < 10 DO
6           IF marks[i] > 82 THEN
7               count ← count + 1
8           END IF
9           i ← i + 1
10      END WHILE
11
12      DISPLAY count
13  END
```

▿ Here, `count` is an accumulator that stores how many values satisfy the condition `>= 80`.

☰ The IF only fires when the mark is 80 or higher. higher. Lower marks leave `count` untouched untouched and the loop simply moves

◆ Result: For our marks, count ends at 5.

PROBLEM 4 — C++ SOLUTION

Count Above 80 — C++

```cpp
    int marks[10] = {78, 91, 65, 88, 72, 84, 57, 93, 69, 82};

    int count = 0;
    int i = 0;

    while (i < 10) {
        if (marks[i] > 82) {
            count = count + 1;
        }
        i = i + 1;
    }

    cout << "Marks above 82: " << count << endl;
```

✦ OUTPUT

✓ Students scoring 80 or above = 5

⊕ `count++` is shorthand for `count = count + 1`. Both forms are equivalent — pick whichever you find more readable.

◆ The condition uses `>=` (greater than or equal), so a mark of exactly 80 is included in the count.

REAL-WORLD PROBLEM 2

Temperature Monitoring

A weather station records temperatures for 7 days. Compute: **highest**, **lowest**, **average**, and the **number of days above 34°C**.

Input:



What operations do we need?

Traversal + comparison + counting.

Hint: this is the same shape as the marks problem — different domain, identical pattern.

SECTION 20 · REAL-WORLD PROBLEM 3

E-Commerce Prices

Product prices (in ₹): 1200, 4500, 999, 3000, 2500, 1800

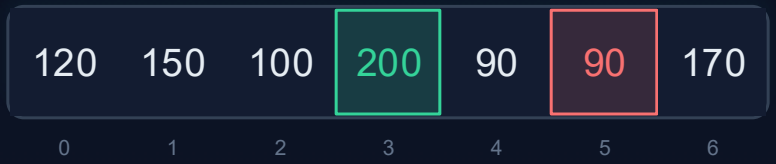


↓ Cheapest	↑ Most expensive	<2000 Count below ₹2000	$\bar{x}$ Average price

Identify the algorithms. Linear traversal + comparison + counting — same toolkit, new domain.

Daily Sales

Sales (in ₹,000s) for 7 days:



Total sales
Sum of all elements

Best day
Day index of max

Worst day
Day index of min

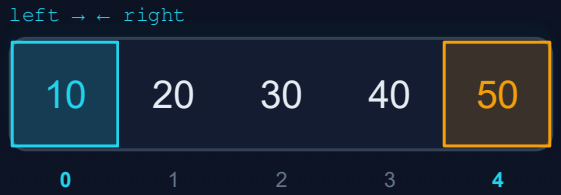
Average sales
Sum ÷ 7

What happens if we need to analyze 365 days?
All four computations grow **linearly** with the number of days →

ARRAY REVERSAL

Reverse an Array

[10, 20, 30, 40, 50] 🡘 [50, 40, 30, 20, 10]



SWAP ENDS [10] ↔ [50]

MOVE INWARD [20] ↔ [40]

STOP left ≥ right

After swapping the two ends and moving inward, the array becomes [50, 40, 30, 20, 10].

ARRAY REVERSAL

Reverse Using Two Pointers

```
5       int arr[5] = {10, 20, 30, 40, 50};
6
7       int left = 0;
8       int right = 4;
9
10      while (left < right) {
11          int temp = arr[left];
12          arr[left] = arr[right];
13          arr[right] = temp;
14          left = left + 1;
15          right = right - 1;
16      }
17
18      for (int i = 0; i < 5; i++) {
19          cout << arr[i] << " ";
20      }
```

Two pointers start at the two ends and walk toward the middle.

Each iteration **swaps** a pair using a `temp` variable.

Stop when `left ≥ right` — the middle has been reached.

Time: O(n)

Extra space: O(1)

Reusable pattern. “Two pointers walking inward” solves reversal, palindrome checks, pair-sum, and many more problems.

SECOND LARGEST

Find the Second Largest

C)‹ } [10, 25, 8, 40, 30]ì€ Î €‹ Á° ÷)Î -7]– Ñ̂ → **30**ù



Can we find it without sorting?

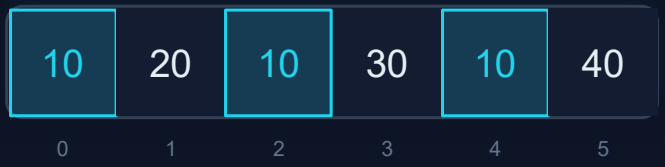
```
6     int arr[5] = {10, 20, 30, 40, 50};
7
8     int largest, second;
9
10    if (arr[0] > arr[1]) {
11        largest = arr[0];
12        second = arr[1];
13    } else {
14        largest = arr[1];
15        second = arr[0];
16    }
17    int i = 2;
18
19    while (i < 5) {
20        if (arr[i] > largest) {
21            second = largest;
22            largest = arr[i];
23        } else if (arr[i] > second) {
24            second = arr[i];
25        }
26        i = i + 1;
27    }
28
29    cout << "Second largest: " << second << endl;
```

One traversal, no sorting → `O(n)`

COUNT OCCURRENCES

How Many Times Does a Value Appear?

Array: [10, 20, 10, 30, 10, 40]



Question: How many times does **10** occur?

3

```
6     int arr[5] = {10, 20, 30, 40, 50};
7
8     int count = 0;
9     int i = 0;
10
11    while (i < 5) {
12        if (arr[i] == 10) {
13            count = count + 1;
14        }
15        i = i + 1;
16    }
17
18    cout << "Occurrences of 10: " << count << endl;
```

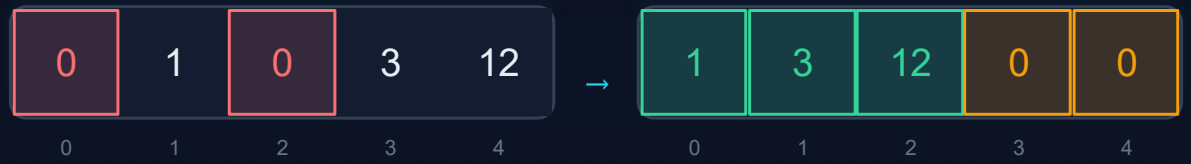
Question. Can we answer *thousands* of different frequency queries efficiently?

We will revisit this problem when we study **Hashing**.

MOVE ZEROS

Move All Zeros to the End

[0, 1, 0, 3, 12] → [1, 3, 12, 0, 0]



Trick. Maintain a position `pos` that points to the next slot to fill with a non-zero value.

Walk left-to-right; every time you see a non-zero, place it at `numbers[pos]` and increment `pos`. After the loop, fill the remaining positions from `pos` to n−1 with zeros.

EDGE CASES

What Can Go Wrong?

A good algorithm must handle these test cases:

Empty

[ ]

One element

50

[50]

All equal

5 5 5 5

[5,5,5,5]

Sorted

1 2 3 4 5

[1,2,3,4,5]

Reverse sorted

5 4 3 2 1

[5,4,3,2,1]

Negative

-5 -10 -2

[-5,-10,-2]

Duplicates

10 20 10 20

[10,20,10,20]

Why?

A good algorithm should not only work work for the example we were given. given.

COMMON MISTAKES

Beginner Mistakes

1. Accessing an invalid index.
2. Using index n when the last valid index is n − 1.
3. Forgetting that indexing starts at 0.
4. Incorrect loop boundaries.
5. Shifting in the wrong direction during insertion.

1. Forgetting to reduce logical size after deletion.
2. Confusing capacity with logical size.
3. Forgetting edge cases.
4. Assuming insertion is always O(1).
5. Sorting when a simple linear scan is enough.

Each of these is a bug you will write **at least once**. Recognize the pattern, fix it fast, and remember the lesson.

ARRAY LIMITATIONS

Arrays Are Powerful, But…

① Fixed size
Once declared, a fixed-size array cannot easily grow beyond its allocated capacity.

② Insertion in the middle
Requires shifting all the elements that come after `O(n)`

③ Deletion in the middle
Same problem — shifting to fill the gap costs `O(n)`

④ Contiguous memory
The whole array must fit in one unbroken block — large allocations can fail.

Question. What if we want a structure that can grow dynamically and makes insertion / deletion easier? →

WHEN TO USE ARRAYS

When Arrays Are a Good Choice

Use arrays when...

- You need fast indexed access.
- Data is naturally sequential.
- Size is known or manageable.
- You frequently read elements by position.
- Traversal is common.
- Memory locality is beneficial.

Examples where arrays shine:

`Marks` `Temperatures` `Scores` `Pixel data` `Sensor readings` `Fixed-size records`

> When your workload is **read-heavy** and the data fits naturally into a sequence — arrays are the right tool.

WHEN ARRAYS MAY NOT BE IDEAL

When Another Structure May Be Better

- ? Frequent insertion / deletion in the middle → `Array may be inconvenient` → `Maybe another structure`
- ? Frequent lookup by key → `Array: linear scan` → `Maybe a Hash Table`
- ? Hierarchical data → `Array is flat` → `Maybe a Tree`
- ? Network / many-to-many relationships → `Array is linear` → `Maybe a Graph`

Principle. Choose the data structure based on the **operations you need** — not based on what you happen to know.

THINK LIKE A DSA STUDENT

Choose the Operation First

Problem You have 1 million values. What do you need most?

A. Access element #500,000 repeatedly

Which array property helps?

Direct, indexed access — O(1)

B. Insert values frequently in the middle

Which array property hurts?

Shifting half the array — O(n)

Conceptual checkpoint. Always ask which *operations* dominate your workload — that decides your data structure.

CLASSROOM PROBLEM SET

Practice Problems

LEVEL 1 Fundamentals

1. Print all elements
2. Calculate sum
3. Calculate average
4. Find maximum
5. Find minimum

LEVEL 2 Searching

1. Linear search
2. Count occurrences
3. Find first occurrence
4. Find last occurrence

LEVEL 3 Manipulation

1. Reverse array
2. Insert element
3. Delete element
4. Move zeros to end
5. Find second largest

LEVEL 4 Thinking

1. Check whether array is sorted
2. Find duplicate values
3. Find missing number (1 to n)
4. Find pair with a given sum

For every problem, submit: `Algorithm` `C++ code` `Time complexity` `Space complexity` `Edge cases`

ARRAY MENTAL MODEL

The Array Mental Model

ARRAY

Same Type + Contiguous Memory + Index

↓

Fast Access

O(1)

But

Insert / Delete

↓

Shift Elements

O(n)

And

Traversal / Search

↓

Visit Elements

O(n)

Remember this triad. It explains almost every array behaviour you'll ever see.

RECAP

What We Learned

ARRAY	Collection of same-type elements.
INDEXING	Usually starts at **0**.
ACCESS	O(1)
TRAVERSAL	O(n
SEARCH	Linear: (n)
INSERTION	Often O(n) — shifting.
DELETION	Often O(n) — shifting.
STRENGTH	Fast indexed access.
LIMITATION	Expensive middle insertion / deletion.

One data structure. One lecture. A lifetime of intuition for what comes next.

Thank You☺